

✓ Project: Smart Fraud Detection Pipeline

Data Profiling and Validation

Author: Snehal A. Bhosale

College: Sanjivani College of Engineering, Kopargaon

Email: snehalbhosale1807@gmail.com

Student ID: CT_CSI_DE_1177

Technology: PySpark, Spark SQL, Parquet/Delta Lake

Objective:

To profile and validate raw financial datasets before processing them through the Bronze, Silver and Gold layers.

✓ Step 1: Import libraries

```
import pandas as pd
import os
```

✓ Step 2: Upload datasets

```
import io
import pandas as pd
from google.colab import files

# Upload files from your computer
uploaded = files.upload()

# Read the uploaded CSV files into DataFrames
accounts = pd.read_csv(io.BytesIO(uploaded['accounts.csv']))
transactions = pd.read_csv(io.BytesIO(uploaded['transactions.csv']))
fraud = pd.read_csv(io.BytesIO(uploaded['fraud_watchlist.csv']))

print('Accounts:', accounts.shape)
print('Transactions:', transactions.shape)
print('Fraud Watchlist:', fraud.shape)
```

No files selected

Upload widget is only available when the cell has been

BROWSE... NO files selected. Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving accounts.csv to accounts.csv
 Saving fraud_watchlist.csv to fraud_watchlist.csv
 Saving transactions.csv to transactions.csv
 Accounts: (505, 5)
 Transactions: (20011, 5)
 Fraud Watchlist: (46, 3)

▼ Step 3: Load datasets

```
accounts = pd.read_csv("accounts.csv")
transactions = pd.read_csv("transactions.csv")
fraud = pd.read_csv("fraud_watchlist.csv")

print("Accounts:", accounts.shape)
print("Transactions:", transactions.shape)
print("Fraud Watchlist:", fraud.shape)
```

Accounts: (505, 5)
 Transactions: (20011, 5)
 Fraud Watchlist: (46, 3)

▼ Step 4: Inspect datasets

```
display(accounts.head())
display(transactions.head())
display(fraud.head())
```

	account_id	customer_name	account_type	credit_limit	branch
0	ACC0059	Customer_59	Salary	500000	Pune
1	ACC0139	Customer_139	Current	100000	Delhi
2	ACC0182	Customer_182	Salary	200000	Pune
3	ACC0302	Customer_302	Savings	500000	Mumbai
4	ACC0254	Customer_254	Savings	200000	Delhi

	txn_id	account_id	txn_date	amount	merchant
0	TXN009314	ACC0332	2025-01-10	107549.32	Swiggy
1	TXN013000	ACC0412	2025-03-17	22297.68	Zomato
2	TXN018018	ACC0486	2025-05-10	115208.63	Zomato
3	TXN013757	ACC0443	2025-07-24	45099.70	ATM_Withdrawal
4	TXN016494	ACC0062	2025-02-25	70884.04	Swiggy

	account_id	fraud_type	flagged_date
0	ACC0116	Account Takeover	2025-05-26
1	ACC0300	Card Skimming	10-Feb-2025
2	ACC0280	Account Takeover	2025-03-03
3	ACC0464	Account Takeover	2025-07-14
4	ACC0072	Account Takeover	2025-05-17

✓ Step 5: Data types

```
print("Accounts")
print(accounts.dtypes)

print("\nTransactions")
print(transactions.dtypes)

print("\nFraud Watchlist")
print(fraud.dtypes)
```

```
Accounts
account_id      object
customer_name   object
account_type     object
credit_limit     int64
branch          object
dtype: object
```

```
Transactions
txn_id          object
account_id      object
txn_date        object
amount          float64
merchant        object
dtype: object
```

```
Fraud Watchlist
account_id      object
fraud_type      object
flagged_date    object
dtype: object
```

✓ Step 6: Missing values

```
print("Accounts Nulls")
print(accounts.isnull().sum())
```

```
print(accounts.isnull().sum())

print("\nTransactions Nulls")
print(transactions.isnull().sum())

print("\nFraud Nulls")
print(fraud.isnull().sum())
```

```
Accounts Nulls
account_id      1
customer_name   1
account_type     0
credit_limit     0
branch          1
dtype: int64

Transactions Nulls
txn_id          0
account_id      1
txn_date        1
amount          2
merchant        0
dtype: int64

Fraud Nulls
account_id      0
fraud_type      1
flagged_date    0
dtype: int64
```

✓ Step 7: Duplicate records

```
print("Duplicate accounts:", accounts.duplicated().sum())
print("Duplicate transactions:", transactions.duplicated().sum())
print("Duplicate fraud records:", fraud.duplicated().sum())
```

```
Duplicate accounts: 0
Duplicate transactions: 1
Duplicate fraud records: 1
```

✓ Step 8: Account ID duplicates

```
print(
    "Duplicate account IDs:",
    accounts["account_id"].duplicated().sum()
)
```

```
Duplicate account IDs: 1
```

✓ Step 9: Orphan accounts

```
account_ids = set(accounts["account_id"].dropna().astype(str).str.strip())

txn_accounts = set(
    transactions["account_id"].dropna().astype(str).str.strip()
)

orphan_transactions = txn_accounts - account_ids

print("Orphan transaction accounts:", orphan_transactions)

Orphan transaction accounts: {'ACC9999'}
```

✓ Conclusion

The raw datasets were successfully profiled and validated. The analysis identified null values, duplicate records, invalid values, inconsistent formatting, and orphan account IDs. These data quality issues will be handled in the Silver layer.